claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-a97fa72f8faac7e9e.jsonl`
- SHA-256: `284c8676d52a7518b4cdf52515cfefdc658cd9bce1a8cfb949cf679e5b652321`
- Source modified: `2026-06-21T19:14:13+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:12:53.036Z)

```
You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the
REAL code the owner will deploy.

CRITICAL repo rules:
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is
checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the
working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared
checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'      (search master's tree)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."        (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you
MAY use Read/Grep/Glob directly there.

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line
evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely.
If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note.
Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat;
'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----
CLAIM (§2.1): `pip install -e .[auth]` installs PyJWT[crypto], and that extra is REQUIRED when
edge auth is on ("else every request 500s").
Verify the engine packaging on origin/master declares an `auth` optional-dependency group
containing PyJWT (with the crypto extra). Check pyproject.toml (and setup.cfg/setup.py if
present): `git -C /c/Users/avidu/Projects/badminton-highlight-indexer show
origin/master:pyproject.toml`. Confirm the extra name is literally `auth` and what
packages/versions it pins. Note if PyJWT is only an extra (not a hard dep) ? that's expected.
```

2. assistant (2026-06-21T19:12:55.421Z)

```
I'll verify this claim about the `auth` optional-dependency group in the engine packaging on
origin/master.
```

3. user (2026-06-21T19:12:57.149Z)

PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).

#

Build backend: hatchling. `backend/` is an intentional NAMESPACE package (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's auto-detection would otherwise miss the un-__init__'d `backend` root.

#

torch/torchvision are deliberately NOT listed in [project.dependencies]: their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621 cannot express. Install them separately ? see [project.optional-depend `gpu` (documentation only) and the README. CI installs CPU torch on its own line.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
```

Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).

```
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0,<0.30.0",
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
]

[project.optional-dependencies]
```

Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).

```
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]
```

M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_a `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/mult

```
auth = ["PyJWT[crypto]>=2.8"]
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a

no-op if absent). Pinned to its git tag per BACKLOG. The private repo is fetched over SSH; if you lack access, simply omit this group ? reporting tests skip via pytest.importorskip. For local dev against a sibling checkout: pip install -e ../sports-obsreport --no-build-isolation

```
reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]
```

`gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot express, so they are NOT declarable here. Install them out-of-band instead: # CUDA 12.4 box: pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org # CPU-only box (also what CI uses): pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```
gpu = []
```

Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs these. Cloud SDKs are imported lazily, so `import backend.storage` works without the install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storag

```
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
```

`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()

```
indexer = "backend.main:main"

[tool.hatch.metadata]
```

The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct reference*); hatchling requires opting in to allow those in metadata.

```
allow-direct-references = true

[tool.hatch.build.targets.wheel]
```

`backend` is a namespace package (no __init__.py); list both import roots explicitly so an editable/wheel install exposes `import backend` and `import training` regardless of cwd.

```
packages = ["backend", "training"]
```

4. user (2026-06-21T19:12:58.426Z)

```
origin/master:pyproject.toml:10:# which PEP 621 cannot express. Install them separately ? see
[project.optional-dependencies]
origin/master:pyproject.toml:43:[project.optional-dependencies]
origin/master:pyproject.toml:51:    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify
(RS256) ? tests + CI
origin/master:pyproject.toml:56:auth = ["PyJWT[crypto]>=2.8"]
```

5. assistant (2026-06-21T19:13:03.660Z)

The claim is fully confirmed. Let me also verify the doc text in DEPLOY_ALPHA.md §2.1 to capture the exact claim wording and check the runtime behavior assertion.

6. user (2026-06-21T19:13:05.776Z)

```
43:pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every
request 500s
65:> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if
`edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud`
is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't
silently come up ? fix what it prints and restart.
```

7. user (2026-06-21T19:13:07.242Z)

```
origin/master:backend/api/auth.py:71:        import jwt
origin/master:backend/api/auth.py:80:        kid = jwt.get_unverified_header(token).get("kid")
origin/master:backend/api/auth.py:84:        claims = jwt.decode(
origin/master:backend/api/auth.py:93:    except Exception as exc:  # jwt.* errors: bad sig /
expired / wrong aud|iss / malformed
```

8. assistant (2026-06-21T19:13:10.621Z)

Let me read the actual auth code to verify the runtime behavior ? does a missing PyJWT cause a 500 when edge auth is on, and does the boot posture "warn" or "fail" on missing PyJWT?

9. user (2026-06-21T19:13:11.743Z)

```
    bypasses the cache so each test is deterministic."""
    if fetcher is not None:
        return fetcher(team_domain)
    now = time.time()
    cached = _jwks_cache.get(team_domain)
    if cached and (now - cached[0]) < _JWKS_TTL_SEC:
        return cached[1]
    jwks = _default_jwks_fetcher(team_domain)
    _jwks_cache[team_domain] = (now, jwks)
    return jwks


def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) -> str:
    """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer + audience) and
    return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY failure."""
    try:
        import jwt
        from jwt import PyJWKSet
    except Exception as exc:  # pragma: no cover - only when the auth extra isn't installed
        raise EdgeAuthError(
            "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
            "(pip install -e .[auth]) on a host with edge_auth_enabled."
        ) from exc

    try:
        kid = jwt.get_unverified_header(token).get("kid")
        if not kid:
            raise EdgeAuthError("token header has no kid")
        signing_key = PyJWKSet.from_dict(jwks)[kid]
        claims = jwt.decode(
            token, signing_key.key, algorithms=["RS256"],
            audience=aud, issuer=_issuer(team_domain),
            options={"require": ["exp", "iss", "aud"]},
        )
    except EdgeAuthError:
```

```
        raise
    except KeyError as exc:  # kid not in the JWKS
        raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
    except Exception as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
        raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc

    user = claims.get("email") or claims.get("sub")
    if not user:
        raise EdgeAuthError("verified token carries no email/sub claim")
    return str(user)


def resolve_tenant(headers: Mapping[str, str], config: Any, *,
                   jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
    """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.

    DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user;
unchanged).
    Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403) on any
miss.
    ``jwks_fetcher`` is injectable for offline tests.
    """
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today

    # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure check
in
    # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured", both
places.
    team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
    if not team_domain or not aud:
        raise EdgeAuthError(
            "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured")
```

10. user (2026-06-21T19:13:13.073Z)

```
origin/master:backend/api/auth.py:10:``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the
base runtime works without it (install the
origin/master:backend/api/auth.py:71:        import jwt
origin/master:backend/api/auth.py:75:            "PyJWT[crypto] is required for edge auth ?
install the 'auth' extra "
origin/master:backend/api/auth.py:76:            "(pip install -e .[auth]) on a host with
edge_auth_enabled."
origin/master:backend/config/startup.py:80:    """True if PyJWT (the ``auth`` extra) is
importable ? probed WITHOUT importing it (find_spec),
origin/master:backend/config/startup.py:82:    (whether PyJWT is installed in the test env must
not change the WARN's presence)."""
origin/master:backend/config/startup.py:130:    # (3) The auth extra (PyJWT[crypto]) is lazy-
imported by auth.verify_cf_access_token. If it's
origin/master:backend/config/startup.py:136:            "security.edge_auth_enabled is ON but
PyJWT[crypto] is not importable ? Cf-Access "
origin/master:backend/config/startup.py:137:            "verification will fail at request time.
Install the 'auth' extra (pip install -e .[auth]).",
```

11. assistant (2026-06-21T19:13:20.998Z)

I now have complete evidence. Let me verify the exact HTTP status code that an `EdgeAuthError` maps to ? the doc says "500s" but the code path may raise an `EdgeAuthError` that gets converted to a 401/403.

12. user (2026-06-21T19:13:23.860Z)

```
origin/master:backend/api/auth.py:30:class EdgeAuthError(Exception):
origin/master:backend/api/auth.py:31:    """A required Cf-Access token was missing or failed
verification ? the caller returns 401/403."""
origin/master:backend/api/auth.py:49:        raise EdgeAuthError(f"unexpected JWKS shape from
{url}")
origin/master:backend/api/auth.py:69:    return the user_id (``email`` else ``sub``). Raises
``EdgeAuthError`` on ANY failure."""
origin/master:backend/api/auth.py:74:        raise EdgeAuthError(
origin/master:backend/api/auth.py:82:            raise EdgeAuthError("token header has no kid")
origin/master:backend/api/auth.py:89:    except EdgeAuthError:
origin/master:backend/api/auth.py:92:        raise EdgeAuthError(f"no JWKS key for kid {exc}")
from exc
origin/master:backend/api/auth.py:94:        raise EdgeAuthError(f"Cf-Access token verification
failed: {exc}") from exc
origin/master:backend/api/auth.py:98:        raise EdgeAuthError("verified token carries no
email/sub claim")
origin/master:backend/api/auth.py:102:def resolve_tenant(headers: Mapping[str, str], config:
Any, *,
origin/master:backend/api/auth.py:107:    Enabled ? require + verify the Cf-Access header; raise
``EdgeAuthError`` (? 401/403) on any miss.
origin/master:backend/api/auth.py:119:        raise EdgeAuthError(
origin/master:backend/api/auth.py:126:        raise EdgeAuthError(f"missing {CF_ACCESS_HEADER}
header (request did not pass Cloudflare Access)")
origin/master:backend/api/uploads.py:20:from fastapi import APIRouter, HTTPException, Request
origin/master:backend/api/uploads.py:71:        raise HTTPException(status_code=400,
detail=str(e)) from e
origin/master:backend/api/uploads.py:74:        raise HTTPException(status_code=400,
origin/master:backend/api/uploads.py:77:        raise HTTPException(status_code=413,
origin/master:backend/api/uploads.py:96:        raise HTTPException(status_code=404,
detail=f"Unknown upload_id '{upload_id}'")
origin/master:backend/api/uploads.py:98:        raise HTTPException(status_code=409,
origin/master:backend/api/uploads.py:115:        raise HTTPException(status_code=413,
detail=overflow + " Upload aborted ? re-init to retry.")
origin/master:backend/api/uploads.py:130:        raise HTTPException(status_code=404,
detail=f"Unknown upload_id '{upload_id}'")
origin/master:backend/api/uploads.py:132:        raise HTTPException(status_code=409,
origin/master:backend/api/uploads.py:153:        raise HTTPException(status_code=404,
detail=f"Unknown upload_id '{upload_id}'")
origin/master:backend/main.py:31:from fastapi import FastAPI, HTTPException, Request
origin/master:backend/main.py:239:from backend.api.job_worker import (  # noqa: F401  (re-
exports: names stay addressable on backend.main)
origin/master:backend/main.py:425:        raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:431:        raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:433:        raise HTTPException(status_code=404, detail=f"No
compiled reel named '{name}'.")
origin/master:backend/main.py:558:@app.post("/api/process", status_code=202)
origin/master:backend/main.py:569:    # spoofed identity header fails here with 401, before any
work) and tags the job's owner_id.
origin/master:backend/main.py:571:        owner_id = edge_auth.resolve_tenant(request.headers,
global_config)
origin/master:backend/main.py:572:    except edge_auth.EdgeAuthError as e:
origin/master:backend/main.py:573:        raise HTTPException(status_code=401, detail=f"edge
auth: {e}") from e
origin/master:backend/main.py:579:        # scheme, which we map to a clean 400 (an unknown
scheme must never 500).
origin/master:backend/main.py:582:        raise HTTPException(status_code=400, detail=str(e))
from e
origin/master:backend/main.py:590:        raise HTTPException(status_code=404, detail=f"Video
file not found at '{req.video_path}'")
origin/master:backend/main.py:593:        raise HTTPException(
origin/master:backend/main.py:594:            status_code=400,
origin/master:backend/main.py:602:        raise HTTPException(status_code=500, detail="Failed to
persist job record.")
```

13. assistant (2026-06-21T19:13:30.745Z)

This is the critical adversarial finding. The doc says "every request 500s" when PyJWT is missing, but the actual code path on origin/master maps a missing-PyJWT failure to a **401**, not a 500.

Tracing the path on origin/master:
- `backend/main.py:571` calls `edge_auth.resolve_tenant(...)`
- `resolve_tenant` ? `verify_cf_access_token` ? on missing PyJWT raises `EdgeAuthError` (`backend/api/auth.py:74-77`)
- `backend/main.py:572-573` catches `edge_auth.EdgeAuthError` and raises `HTTPException(status_code=401, ...)`

So when edge auth is on and PyJWT is not installed, requests get a clean **401** ("edge auth: PyJWT[crypto] is required..."), not a 500. The packaging claim itself (the `auth` extra exists and pins `PyJWT[crypto]>=2.8`) is fully confirmed; the "else every request 500s" justification is technically inaccurate.

Let me confirm there's no earlier code path that would 500 ? e.g., whether `resolve_tenant` is the first thing that touches jwt on every request.

14. user (2026-06-21T19:13:34.164Z)

```
origin/master:backend/api/auth.py:30:class EdgeAuthError(Exception):
origin/master:backend/api/auth.py:49:        raise EdgeAuthError(f"unexpected JWKS shape from
{url}")
origin/master:backend/api/auth.py:69:    return the user_id (``email`` else ``sub``). Raises
``EdgeAuthError`` on ANY failure."""
origin/master:backend/api/auth.py:74:        raise EdgeAuthError(
origin/master:backend/api/auth.py:82:            raise EdgeAuthError("token header has no kid")
origin/master:backend/api/auth.py:89:    except EdgeAuthError:
origin/master:backend/api/auth.py:92:        raise EdgeAuthError(f"no JWKS key for kid {exc}")
from exc
origin/master:backend/api/auth.py:94:        raise EdgeAuthError(f"Cf-Access token verification
failed: {exc}") from exc
origin/master:backend/api/auth.py:98:        raise EdgeAuthError("verified token carries no
email/sub claim")
origin/master:backend/api/auth.py:102:def resolve_tenant(headers: Mapping[str, str], config:
Any, *,
---- all HTTPException status mapping for edge auth in main ----
    """
    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
    # spoofed identity header fails here with 401, before any work) and tags the job's owner_id.
    try:
        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root", None)
    try:
```

15. assistant (2026-06-21T19:13:37.089Z)

Evidence is complete and conclusive. The packaging claim is confirmed; the runtime justification ("else every request 500s") is refuted by the code, which maps the missing-PyJWT failure to a 401.

16. user (2026-06-21T19:13:53.781Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 3042 bytes): {"claim": "§2.1: `pip install -e .[auth]` installs PyJWT[crypto], and that extra is REQUIRED when edge auth is on (\"else every request 500s\").",

"id": "deploy-alpha-2.1-auth-extra", "verdict": "part
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>

17. assistant (2026-06-21T19:13:56.165Z)

I had a malformed JSON (evidence should be an array, not a stray string). Let me retry with a
clean structure.

18. user (2026-06-21T19:14:08.532Z)

Structured output provided successfully

19. assistant (2026-06-21T19:14:13.157Z)

Verified. The packaging claim is confirmed; the failure-mode wording is wrong (401, not 500).
Summary recorded via StructuredOutput.

Key findings:
- `auth` extra exists and pins `PyJWT[crypto]>=2.8` ? engine `origin/master:pyproject.toml:56`,
under `[project.optional-dependencies]` (line 43), and it's an extra not a hard dep (expected).
- Discrepancy: doc says a missing extra makes "every request 500s", but `origin/master` maps the
missing-PyJWT `EdgeAuthError` to a **401** (`backend/api/auth.py:71-77` ?
`backend/main.py:571-573`). Severity minor ? the deploy fails closed (safe-but-unusable), not
crashing. Suggested doc fix: "else every request is rejected 401".